

How to Do Spec-Driven Development

The workflow AI-native teams use to build complex software. This is how they do it!



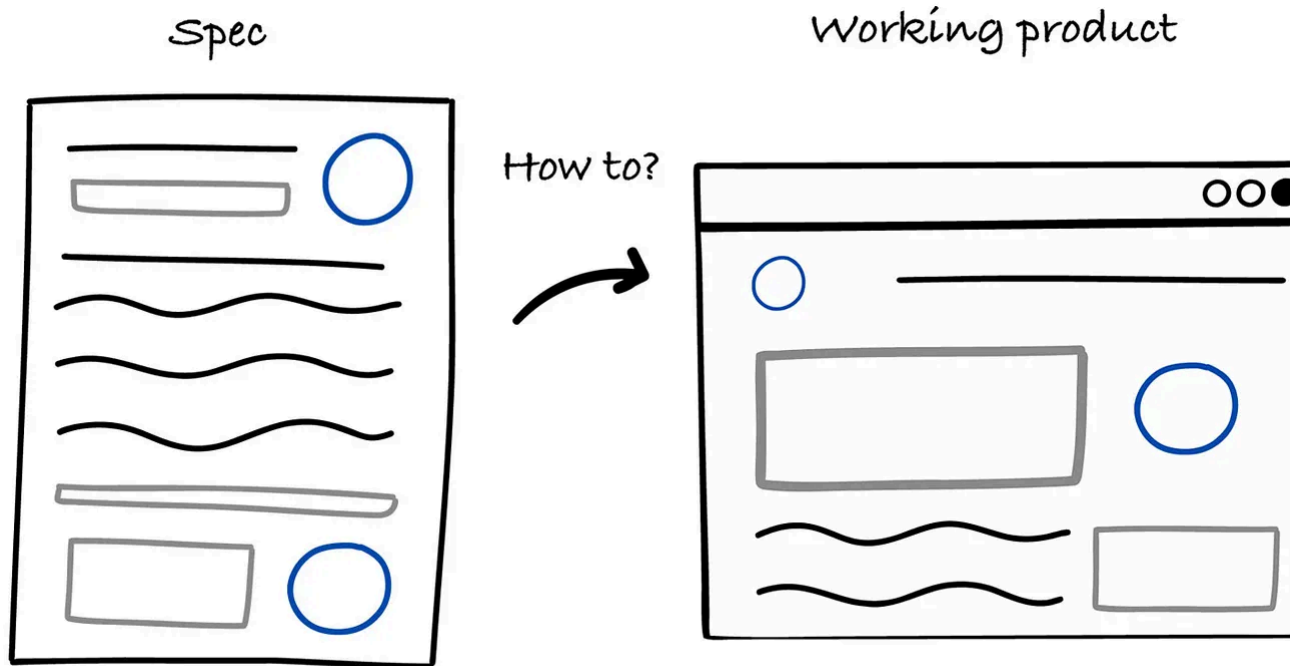
GREGOR OJSTERSEK

JUL 27, 2026 • PAID

♡ 233

💬 24

🔄 42



This week's newsletter is sponsored by [Unblocked](#).

[Webinar] Can you prove AI is working?

AI is in your engineering workflow. While the token spend shows it, the throughp doesn't. The human is very much still in the loop, and that's a context problem.

WEBINAR

Unblocked

Can you prove AI is working?

AUG 19, 2026 | 9 AM PT | 12 PM ET

AI ACTIVITY

REWORK

PRODUCTIVITY

[Join live on Aug 19 \(FREE\)](#) to learn:

- The 4 metrics to measure the gap where gains leak out before production.
- The 8 stages of context maturity, the specific walls capping your metrics, and a free tool to pinpoint where your team is.
- Why more MCPs and bigger context windows aren't enough, and what it takes to get real value from your agents.

Thanks to Unblocked for sponsoring this newsletter. Let's get back to this week's thought!

Intro

Earlier this year, I was in San Francisco talking with different companies, and one of them was [Larridin](#). Funnily, I came right in the middle of their team meeting (sorry guys).



They are building a platform for what many companies are wondering these days: how to measure AI adoption and AI fluency.

Things that especially stood out from our conversations:

- They are a true definition of an AI-native team, with AI being at the core of everything they do.
- They are big advocates of spec-driven development. The spec needs to be almost perfect before they start with the implementation.
- Every engineer owns the feature end-to-end → from creating the spec to implementing it, to ensuring it's working correctly.
- [Ameya Kanitkar, CTO, Larridin](#), mentioned they also like to work with more junior level engineers, but they need to leave and breathe AI and always look to improve and get better in this area.

- A great analogy about using AI → You have a fast-driving car right in front of why not use it?

I've talked with the devs, and also with Ameya, whose insights I am sharing today! today's article, we'll dive into how they are doing spec-driven development at La

Let's start!

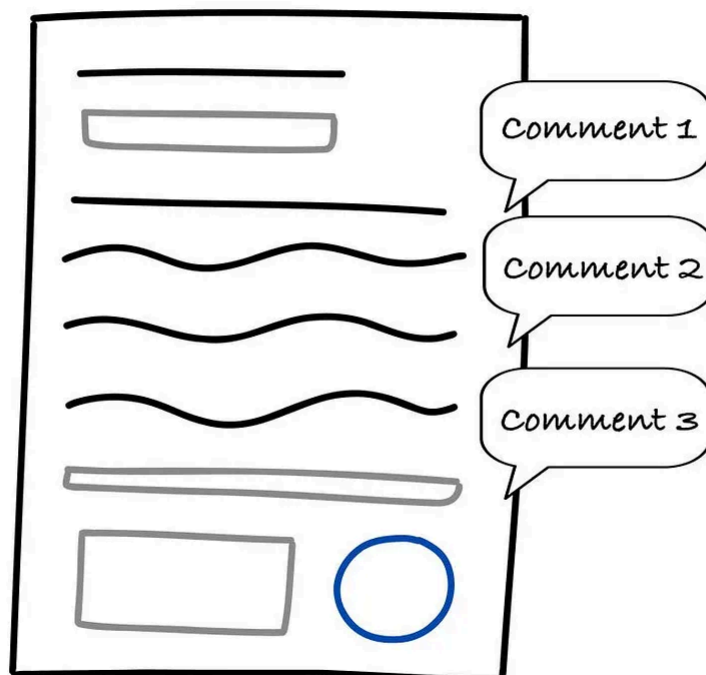
What is spec-driven development?

There are many different definitions out there, but here is what it means for me:

Spec-driven development is a way of building software where you focus on building a good specification first, and then use AI to implement the project.

There are many ways you can write the spec, for example, Michael Bolin, Codex Lead, uses Notion to create a spec and then asks all the other engineers on the team to take a look and provide comments.

Tech spec in Notion



This helps spot any issues beforehand and improves the overall implementation by the LLMs. To learn Michael's full process for AI-assisted engineering, read this article:



How OpenAI Codex Tech Lead Does AI-Assisted Engineering

GREGOR OJSTERSEK • JUN 4

[Read full story →](#)

The popularity behind spec-driven development increasing

I am seeing spec-driven development as the way to build software used by more more companies, and it definitely makes sense.

Especially with the cost of tokens just keep increasing, you don't have the luxury just infinitely prompting in order to get the desired output (unless you work with unlimited tokens).

With a great spec, you can usually use a lower-tier model and get a very similar output, the reason for that is that the instructions in a great spec are very clear.

So, this is also the workflow that I see many companies use:

- Write a spec (with the help of higher-tier LLMs)
- Build the spec (with the lower-tier LLMs)

Now, let's get into more details on how Larridin does spec-driven development!

1. Start with a spike first

Ameya mentioned that before they start writing the spec, they prove that the "ris things work.

In software development, there are a lot of unknowns when you are building thin and it's impossible to write a complete spec if you are dealing with unknowns.

So, what they do is, they create a [spike](#) task to resolve these unknowns of a spec project first.

The goal is not to build a full MVP, it can be a small, ugly, throwaway-quality work enough to answer the question "does this actually work"?

The purpose is not to extend it afterward, as you would normally do with the MVP the goal is to remove any technical uncertainty, which, alternatively, would just contaminate the spec with various different options.

Some examples:

- If the API that you're going to build has 3 edge cases, spike those.
- If your whole system depends on how fast your API is, spike the hot path and measure it.
- If you're building a new view that has a lot of records in it, prove that the API return the results fast enough.

And another example that Ameya shared from their work:

We wanted to create a model to measure "AI Fluency". So we wrote a spike to ensure our core logic works. Once we were happy with the results, the rest of design followed (queue, monitoring, database, etc.). Spec was simply for ensuring core logic works and is token efficient.

The goal is to stop when you have an answer, not when the code is pretty. If you're worried about naming variables, functions, and file structure, stop, that's a sign you're putting too much thought into implementation details.

2. Promote the spike to reference implementation

Once the spike works, don't throw it away. It's now your reference implementation

This is now a reference that you can point the LLM to and say "I know this pattern works, because here it's working!"

Ameya believes that this is one of the most underrated things when you're doing spec-driven development. A working example is worth a lot more than a full page of the spec. That's how the model stops hallucinating and stops "guessing" the behavior as well.

The reason is that the model can now check the reference and see the approach that has been taken to implement that specific part.

The reference implementation belongs in the repo in a known location. All of the references are in a location: *spec/spikes/reference/referenceName*.

And also another tip here is to comment on the specific parts that prove the approach and also add comments to the parts where things are hardcoded, maybe the error handling is missing, or the implementation is not on point.

The more info you give in the codebase, the more accurate the overall implementation by the LLM is going to be.

Ameya believes that a spike without comments can be similarly dangerous as not doing it at all, because the model can just reproduce the wrong parts alongside the right ones.

3. Write the spec

Now that you have the reference implementation with comments, it's time to write the spec! The spec should be detailed enough to hand to a junior engineer or a small model.

You can and should, of course, use AI to help you write the spec. When you use AI to write the spec, Ameya suggests forcing the model to restate what needs to be done, list any assumptions, and also flag any kind of ambiguity.

So, in case there are any non-handled assumptions and ambiguity left, you can either create a new spike or update the existing one.

Here is also a desired spec structure that they use at Larridin:

- **Problem statement at the top**

What you're solving and for whom. One paragraph.

- **Non-goals**

What this system explicitly will not do. This is where you head off half the overengineering before it starts.

- **Assumptions**

The details that you have already decided, so the model doesn't "silently" pick the Cursor or offset pagination? Lazy loading of components? You need to write all of these down.

- **Reference implementation**

Link to the spike. Call out (if not already in the comments) which parts are shortcuts and which parts prove the implementation.

- **Architecture**

Data model, module boundaries, interfaces, error handling, etc.

- **Test plan**

What unit, integration, and/or e2e tests are needed and how they need to be done. More on this in the next phase.

Example of a good spec shared by Ameya: [Full example of the spec](#).

And a recommendation he shared:

My recommendation is to write the smallest spec that unambiguously specifies the system. If a section feels like fluff, cut it. If a decision feels obvious, write it down anyway. "Obvious to you" is not the same as "obvious to the model".

4. Write/adjust the test plan before the implementation plan

This post is for paid subscribers

[Subscribe](#)

Already a paid subscriber? [Sign in](#)

© 2026 Gregor Ojstersek · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture